

Predicting Cloud Infrastructure Misconfigurations Using Logistic Regression Analysis of Azure Resource Templates

Brad Wheeler

December 2025

Abstract

Cloud infrastructure misconfigurations represent a significant security and operational risk, with Gartner estimating that 99% of cloud security failures result from customer misconfigurations. This study applies logistic regression analysis to a synthetic dataset of 13,000 template-level observations derived from over 1,000 Microsoft Azure Bicep resource templates to determine which configuration properties are statistically significant predictors of misconfigurations. The analysis identified 26 statistically significant predictors out of 40 features examined, with the model achieving 89.7% accuracy and ROC AUC of 0.962. Key findings indicate that enabling encryption ($OR = 0.07$) and automatic patching ($OR = 0.08$) reduce misconfiguration odds by over 90%, while disabling versioning ($OR = 10.89$) or soft delete ($OR = 10.47$) increases risk approximately tenfold. Template composition variables were not significant predictors, suggesting configuration choices matter more than template complexity. These findings provide an evidence-based framework for prioritizing cloud security remediation efforts.

Keywords: *cloud security, infrastructure-as-code, logistic regression, Azure, misconfiguration detection, risk quantification*

1. Introduction and Research Question

Organizations that require cloud infrastructure must configure, manage, and maintain hundreds of properties across multiple resource types to keep their technical infrastructure safe, secure and reliable. With the fast pace of development, the increasing number of resources deployed, and high churn rates as part of DevOps practices, managing configuration across large distributed systems is becoming increasingly burdensome for IT departments across the globe. Improper configurations lead to security vulnerabilities, compliance failures, and operational issues. According to Gartner estimates, 99% of cloud security failures result from customer misconfigurations, with the average data breach costing \$4.35 million (IBM, 2023). While rule-based static analyzers can detect known misconfiguration patterns, they provide only binary outputs without risk quantification. Security teams need to not only detect but also know which configurations present the greatest risk so remediations can be prioritized and pursued efficiently.

Based on this industry challenge, the central research question being asked is:

Which template-level Azure resource configuration properties, such as the presence of any publicly accessible storage, the proportion of VMs with secure boot enabled, or whether soft delete is universally configured, are statistically significant predictors of cloud infrastructure misconfigurations across security, operational, and reliability domains?

The study applies logistic regression analysis to a synthetic dataset of 13,000 template-level observations derived from over 1,000 Microsoft Azure Bicep resource templates. Rather than analyzing individual resource properties, this analysis aggregates configurations at the template level to examine patterns such as “any storage account lacking soft delete” or “percentage of VMs with secure boot enabled.” This aggregation transforms deterministic rule outputs into probabilistic relationships suitable for statistical modeling. The primary goal is determining whether template-level configuration patterns can meaningfully predict misconfigurations, with risk magnitude quantified through odds ratios. The study also includes supplemental chi-square and ANOVA tests to examine whether misconfiguration rates differ significantly across mutation scenario categories (security, operational, and reliability).

1.1. Hypotheses

The null hypothesis is:

H0: Azure template resource configuration properties (public access settings, TLS version, encryption status, secure boot, boot diagnostics, patch management, versioning, soft delete, availability sets, managed disks) are not statistically significant predictors of infrastructure misconfigurations.

The alternative hypothesis is:

H1: Azure template resource configuration properties (public access settings, TLS version, encryption status, secure boot, boot diagnostics, patch management, versioning, soft delete, availability sets, managed disks) are significant predictors of infrastructure misconfigurations.

To test these hypotheses, the study uses logistic regression with L2 regularization and a 70/30 segmented train/test split. The null hypothesis will be rejected if the model successfully exceeds the accuracy by at least 5 percent ($> 78.6\%$) and achieves ROC AUC above 0.80, with significance assessed at $\alpha = 0.05$. For individual predictors, risk magnitude is quantified using odds ratios with 95% bootstrap confidence intervals—any predictor whose interval excludes 1.0 is considered significant. Chi-square tests check whether scenario category and misconfiguration status are independent, while one-way ANOVA compares mean misconfiguration counts across categories. Effect sizes are reported using Cramér’s V and eta-squared (η^2).

2. Data Collection

The data for this study was generated using CRUX (Cloud Resource mUtation eXaminer), an open-source tool developed by the author to analyze Azure infrastructure-as-code templates.

CRUX processes Bicep and ARM templates from Microsoft’s Azure Quickstart Templates repository, a collection of over 1,000 community- contributed, Microsoft approved and curated deployment templates representing real-world Azure infrastructure patterns. Each template defines one or more Azure resources with specific configuration properties. The dataset employs a controlled experimental design with mutation scenarios. Templates are first analyzed in their baseline state as originally authored, then systematically mutated to introduce specific categories of misconfigurations based on defined industry standards. This approach creates a ground-truth dataset where misconfiguration labels are deterministic and verifiable against the CIS Microsoft Azure Foundations Benchmark v2.0.0, which is the industry- standard security framework selected for this study.

The template-level aggregation methodology transforms resource- level properties into template-level features. Rather than treating each resource as an independent observation (which would create redundant relationships between properties and labels), features are aggregated across all resources within a template. For example, indicates whether any storage account in the `any_public_access` template allows public blob access, while represents `pct_secure_boot` the percentage of virtual machines with secure boot enabled. This aggregation creates probabilistic relationships suitable for statistical modeling.

The original dataset contained 14,000 observations across 14 scenarios; however, a data generation anomaly was discovered during review which required exclusion of scenarios `all_mutations` from the sample dataset. The redaction reduced the usable data to ~13,000 usable observations across 12 template mutation scenarios. After redactions, the data set still exceeded the minimum requirements of the study by 85% (>7000).

The total study includes 36 independent variables representing configuration properties (public access settings, TLS version, encryption status, secure boot, boot diagnostics, patch management, versioning, soft delete, availability sets, managed disks, and others) and a binary dependent variable (: 0/1). The `has_any_misconfiguration` dependent variable had a positive rate of 73.6%, indicating sufficient variance for logistic regression analysis.

2.1. Mutation Scenario Architecture

Table 1 presents the reference scenario architecture for CRUX data generation.

Category	Scenario	Description	Mutations Applied
CONTROL	baseline	No mutations; templates as authored	None
SECURITY	<code>security_high</code>	High-severity security mutations	Public blob access, HTTP allowed, open NSG rules, no encryption

Category	Scenario	Description	Mutations Applied
SECURITY	<code>security_medium</code>	Medium-severity security mutations	Weak TLS, no secure boot, no vTPM
SECURITY	<code>security_all</code>	All security mutations	Combined high + medium
OPERATIONAL	<code>operational_high</code>	High-severity operational mutations	No boot diagnostics, no auto-patching
OPERATIONAL	<code>operational_medium</code>	Medium-severity operational mutations	No versioning, no soft delete, no managed identity
OPERATIONAL	<code>operational_all</code>	All operational mutations	Combined high + medium
RELIABILITY	<code>reliability_high</code>	High-severity reliability mutations	No availability set, unmanaged disks
RELIABILITY	<code>reliability_medium</code>	Medium-severity reliability mutations	No DDoS protection, no service endpoints
RELIABILITY	<code>reliability_all</code>	All reliability mutations	Combined high + medium
COMBINED	<code>security_operational</code>	Security + Operational	All security + all operational
COMBINED	<code>security_reliability</code>	Security + Reliability	All security + all reliability
COMBINED	<code>operational_reliability</code>	Operational + Reliability	All operational + all reliability

See Table 1 (in the Tables Appendix) for the full Reference Scenario Architecture used for CRUX data generation.

2.2. Advantages of Methodology

The CRUX-generated synthetic dataset provides ground-truth labels through controlled mutations. Unlike observational studies where misconfiguration status may be unknown or inconsistently labeled, each mutation scenario produces deterministic, verifiable labels based on CIS benchmark rules. The inclusion of a baseline control group (templates in their original, unmutated state) enables direct comparison between properly configured and misconfigured templates which established causality between configuration properties and misconfiguration outcomes.

Additionally, the use of a synthetic dataset methodology offers significant cost advantages over alternatives. Deploying thousands of template-scenario combinations to live Azure infrastructure

would incur substantial compute costs (and perhaps introduce real security risks) and manual labeling by security experts would require hundreds of hours, potentially at professional consulting rates. By analyzing templates statically and deriving labels programmatically from CIS benchmark rules, this study achieves comparable analytical rigor at a fraction of the cost making the methodology accessible to organizations and researchers without extensive cloud budgets. The methodology is fully reproducible, open-source, and incurs zero deployment costs.

2.3. Limitations of Methodology

This study has several limitations. Synthetic data generation may not capture the full complexity of real-world deployment templates, which often contain custom configurations, third-party modules, and organizational-specific patterns. The data set is derived from Microsoft reference architectures rather than producing actual production templates, which may underestimate the full complexity of real world misconfiguration scenarios, non-template based configuration factors, or organizational specific patterns. Additionally, the baseline scenario showing 0% misconfigurations indicates these reference templates are well-maintained, which may not reflect internet open-source realities.

2.4. Methodological Challenges

There were two significant challenges that required solutions to enable this study. The original study design analyzed single-resource templates at the individual resource level, which created a fundamental problem: the relationship between configuration properties and misconfiguration labels was redundant rather than statistical. When a single storage account's property both defines the independent variable and `allowBlobPublicAccess` determines the misconfiguration label, making the prediction circular by definition and not aligning with the long-term goals of the study. Iterative refinements determined that compound deployment scenarios (templates containing multiple interdependent resources) transformed deterministic rule-based relationships into probabilistic ones suitable for statistical modeling.

Aggregating to the template level using patterns like `or any_public_access` transformed deterministic rule-based relationships into `pct_secure_boot` genuine probabilistic ones suitable for logistic regression. A template with `may` or `may not` exhibit misconfigurations depending `any_public_access = 1` on other configuration factors, resource composition (such as), and mutation scenario enabling legitimate statistical `private_link_enabled` inference.

Second, the initial data generation produced 14 mutation scenarios, but one scenario () exhibited anomalous behavior during exploratory `all_mutations` data analysis. Investigation revealed a bug in the mutation reference resolution system: the scenario was configured to apply all `all_mutations` mutations via nested references (, , `security_all operational_all`), but the reference resolution function failed to correctly `reliability_all` expand these references,

resulting in zero mutations being applied despite . An issue was filed with the GitHub project and the problematic `is_mutated=1` data was excluded, reducing the dataset by 1,000 observations. The final dataset included 13,000 observations exceeding the minimum requirement for the study.

3. Data Extraction and Preparation

To support the study, data extraction and preparation was performed by copying reference CSV data (raw) from the CRUX GitHub project, then using the Python programming language inside of a Jupyter Notebook for exploratory data analysis and artifact rendering. The following libraries were used to support the process: Pandas was used as a general purpose foundation for data handling and manipulation (McKinney, 2017), NumPy was included to support specific numerical computations, scikit-learn provided train/test split capabilities; however, statsmodels (Seabold & Perktold, 2010) was used in place of scikit-learn LogisticRegression library, due to its statistical output strengths, ease-of-use, and familiarity. Visualizations implemented matplotlib and seaborn to render correlation heatmaps, confusion matrix, and styled plots as visual aids.

The data extraction process required loading the CSV dataset generated by CRUX into a pandas DataFrame. This resulted in 14,000 usable observations with 59 potential feature variables. Initial inspection confirmed the dataset contained complete records for all expected mutation scenarios and combinations of 1,000 baseline templates. During preparation, the problematic scenario was redacted which showed zero `all_mutations` misconfigurations and was confirmed to be a dataset bug. Data quality checks confirmed that no missing values, imputation or deduplication was required for remaining data. Of the columns available, 14 fields were found unsuitable for analysis since they represent internal processing metadata or summary data for mutation scenarios and were redacted from the working dataset. These exclusions produced a working dataset of 13,000 rows and 45 feature variable columns.

Exploratory analysis examined the dependent variable distribution, which had a 73.6% positive rate (percentage of misconfigured samples) and 26.4% negative rate (properly configured samples with dependent variable = 0), with the baseline scenario being entirely negative cases as expected for the control group. Outlier analysis was performed using the IQR method across all independent variables. Notable outlier rates were found in `count_vm` (12.0%), (17.5%), and (6.9%). These `count_nsg dependency_density` outliers were retained because: (1) template complexity naturally varies and represents legitimate deployment patterns, (2) the values are not errors but real infrastructure configurations, and (3) logistic regression is robust to outliers in predictor variables (Hosmer et al., 2013). Multicollinearity assessment via correlation matrix identified three feature pairs with correlations exceeding 0.90, including a perfect inverse correlation between `all_managed_disks` and `any_unmanaged_disk`; one feature from each highly correlated pair was `any_unmanaged_disk` removed to prevent coefficient instability in the logistic regression model. The dataset was then split into training (70%, n=9,100) and test (30%, n=3,900) sets using stratified sampling to preserve the class distribution in both partitions.

Finally, continuous features were standardized using z-score normalization (subtracting the mean and dividing by standard deviation) fitted on the training set and applied to both sets, ensuring features contributed equally to the model regardless of their original scales.

3.1. Data Loading and Initial Inspection

Import necessary libraries and load the dataset into a pandas DataFrame, then inspect the first few rows and data types to understand the structure.

```
# Core libraries import pandas as pd import numpy as np import json import warnings warnings
# Visualization
import matplotlib.pyplot as plt import seaborn as sns plt.style.use('seaborn-v0_8-whitegrid')
# Statistical analysis
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import ( accuracy_score, precision_score, recall_score, f1_score, roc_auc_score)

from statsmodels.stats.outliers_influence import variance_inflation_factor
print("Libraries have loaded successfully")
print(f"pandas version: {pd.__version__}")
print(f"numpy version: {np.__version__}")
```

Load the data set and inspect the shape and columns. Assertions are included to confirm the data loaded correctly.

```
# Load the template-level dataset
df = pd.read_csv('template_level_data.csv')
print(f"Dataset shape: {df.shape}")
print(f"Columns: {df.columns}")
# Assert DF is > 7000 rows and has > 50 columns
assert df.shape[0] > 7000, "Dataset has fewer than 7000 rows!"
assert df.shape[1] > 50, "Dataset has fewer than 50 columns!"
```

Result:

Dataset shape: (14000, 59) Columns: Index(['template_id', 'template_name', 'scenario_id', 's

3.2. Scenario Filtering

One scenario with a data generation bug was excluded from the dataset to ensure data integrity. The scenario was found to have zero `all_mutations` misconfigurations due to a bug in the mutation reference resolution system.

This scenario was redacted from the working dataset.

```
# Exclude scenario with mutation resolution bug
excluded_scenarios = ['all_mutations']
df_clean = df[~df['scenario_id'].isin(excluded_scenarios)]
print(f"Observations after filtering: {len(df_clean):,}")
print(f"Scenarios retained: {df_clean['scenario_id'].nunique()}")
```

result

Observations after filtering: 13,000 Scenarios retained: 13

3.3. Dependent Variable Distribution

The dependent variable was reviewed to determine `has_any_misconfiguration` the distribution of misconfigured versus well-configured templates scenarios.

The positive rate (misconfigured) was 73.6% and the negative rate (properly configured) was 26.4%.

```
# DV distribution
dv_counts = df_clean['has_any_misconfiguration'].value_counts()
print(f"Positive cases (DV=1): {dv_counts[1]:,} ({dv_counts[1]/len(df_clean):.1%}")
print(f"Negative cases (DV=0): {dv_counts[0]:,} ({dv_counts[0]/len(df_clean):.1%}")
```

result

Positive cases (DV=1): 9,563 (73.6%) Negative cases (DV=0): 3,437 (26.4%)

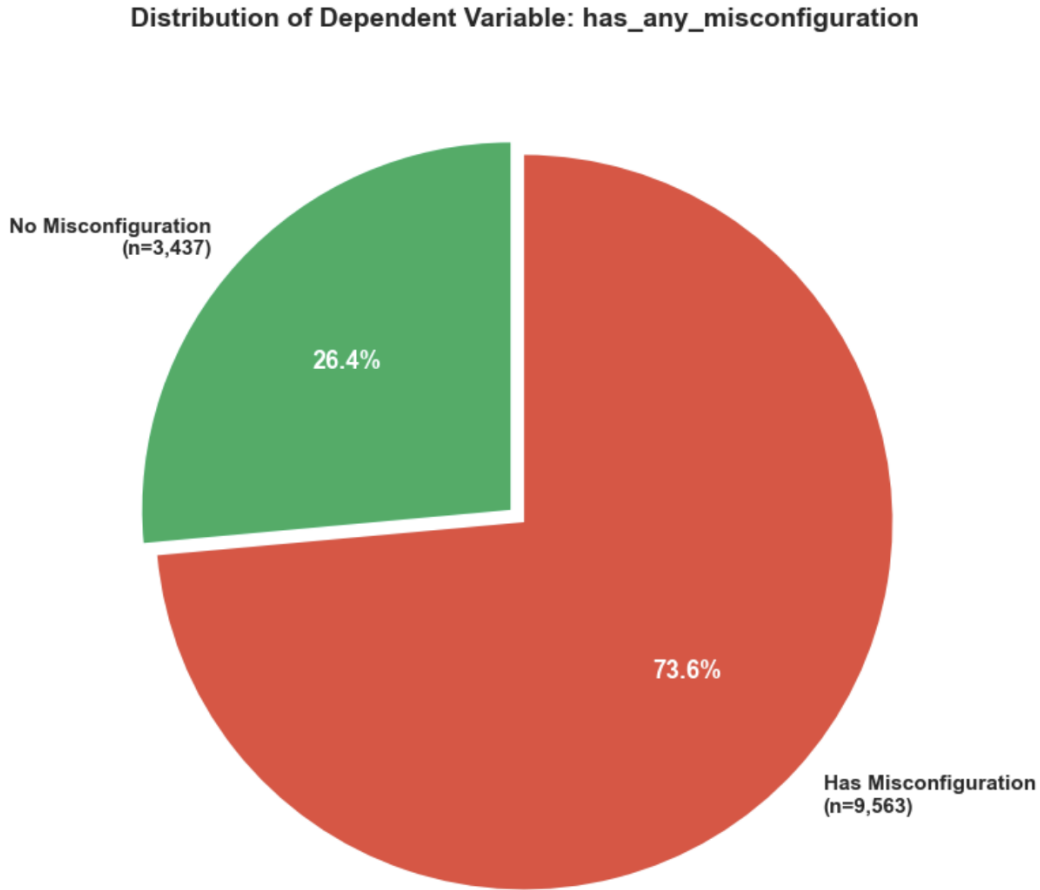


Figure 1: Figure 1

Figure 1: Distribution of Dependent Variable: `has_any_misconfiguration`

3.4. Misconfiguration Rate by Scenario Category

Misconfiguration rates by each mutation scenario were calculated to show how mutation classes are distributed across the data set. Note that baseline scenarios will always show 0% misconfigurations since they are unmutated templates as authored; however, that is not an

indicator these templates are perfect real-world examples of secure and reliable operational states, just that they are unmutated as designed and curated by Microsoft to cover standard use cases.

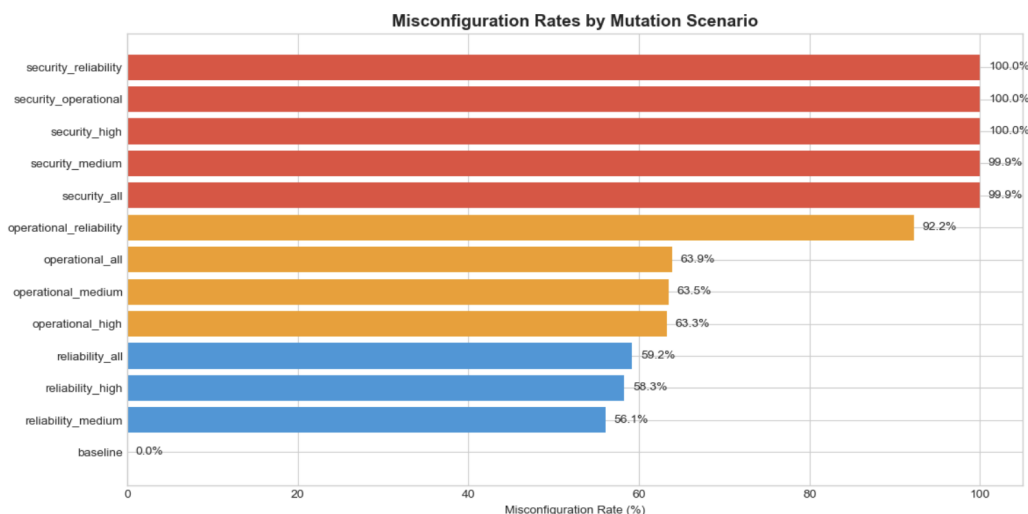


Figure 2: Figure 2

Figure 2: Misconfiguration Rates by Mutation Scenario

3.5. Outlier Analysis

The interquartile range (IQR) method was applied to identify outliers in continuous independent variables. Variables with extreme values were noted, but outliers were retained since they represent legitimate complex deployment patterns not data errors. Notable outlier rates were found in `count_vm` (12.0%), (17.5%), and (6.9%). These outliers `count_nsg` `dependency_density` were retained because: (1) the template complexity naturally varies and represents legitimate deployment patterns, (2) the values are not errors, just real infrastructure configurations, and (3) logistic regression is hypothetically robust to outliers in predictor variables (Hosmer et al., 2013).

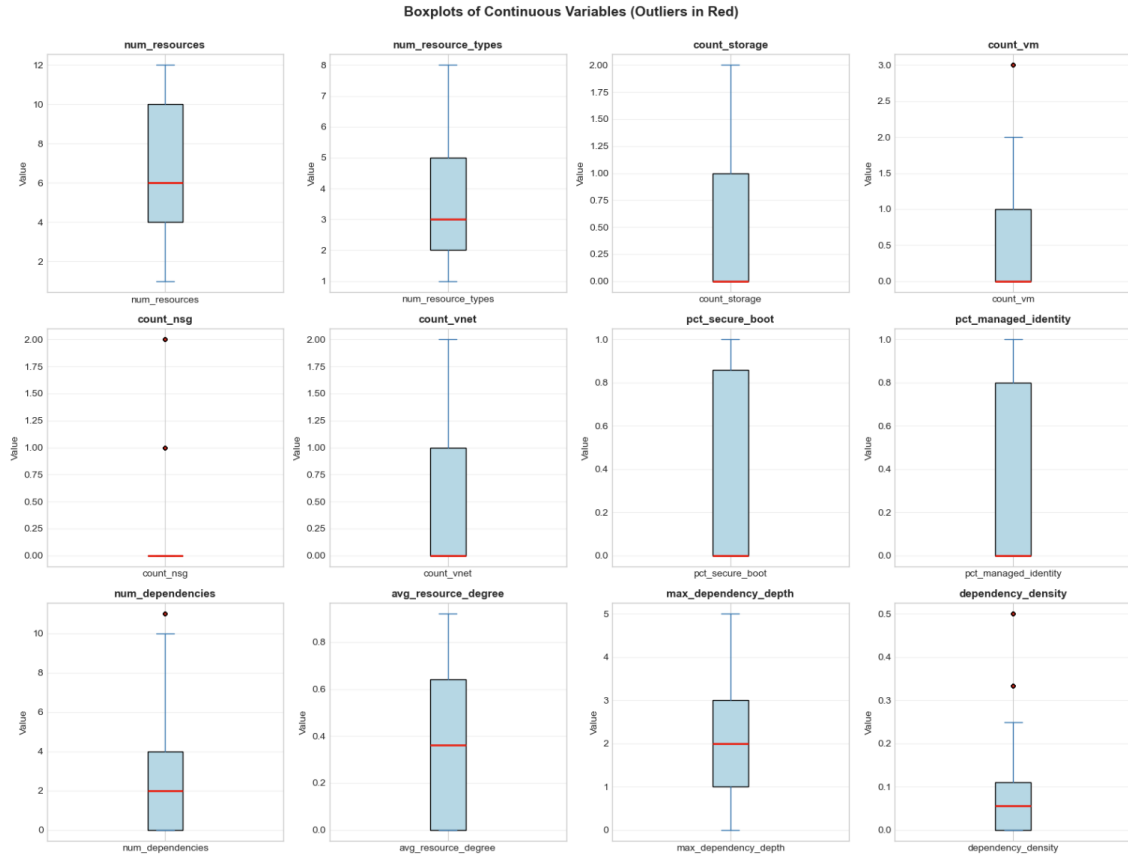


Figure 3: Figure 3

Figure 3: Boxplots of Continuous Variables (Outliers in Red)

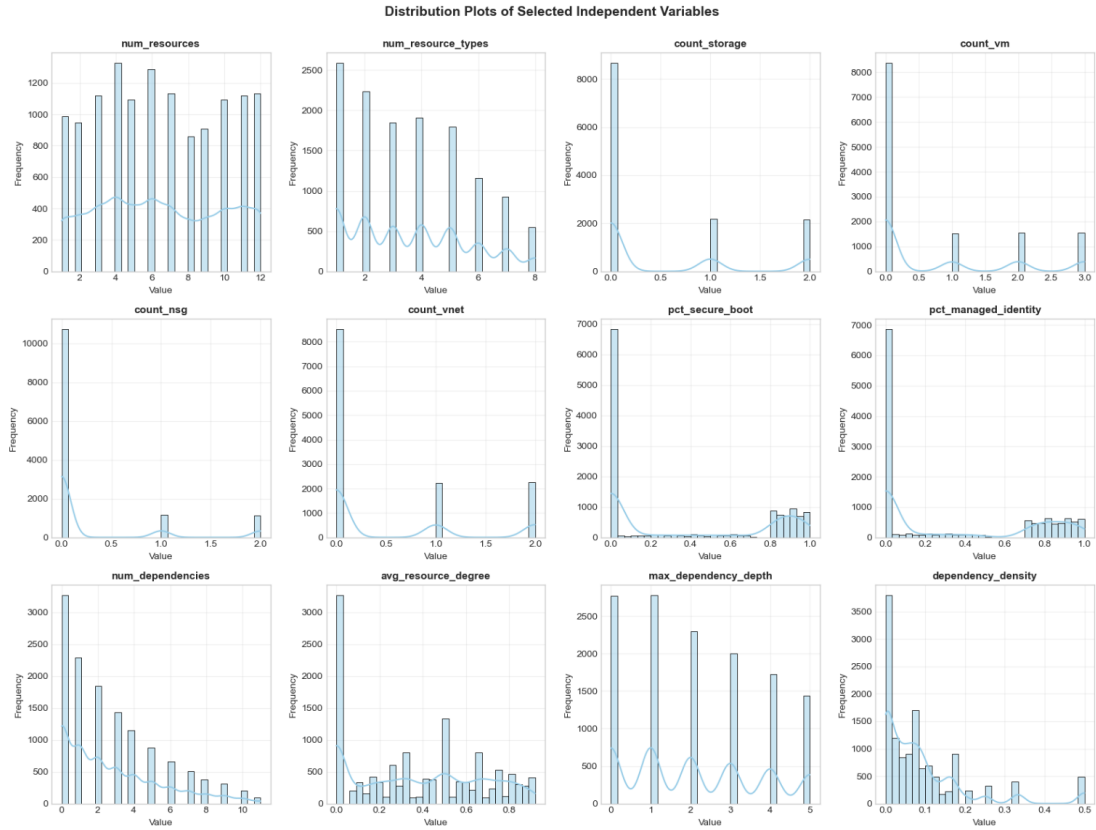


Figure 4: Figure 4

Figure 4: Distribution Plots of Selected Independent Variables

3.6. Bivariate Analysis

Before fitting the logistic regression model, bivariate correlations with the independent variable and the dependent variable were reviewed to identify any strong relationships that may inform feature selection or engineering.

Positive correlations: ($r = 0.48$) shows the strongest positive `is_mutated` correlation, which validates the experimental design—mutated templates should exhibit more misconfigurations than baseline templates. VM-related features (`has_vm`, `any_diagnostics_disabled`, `pct_secure_boot`, `count_vm`) cluster among the top predictors, suggesting virtual machine configurations are a primary source of misconfiguration risk. Network security indicators (

, ,) also show moderate `has_nsg` `any_ddos_disabled` `any_no_service_endpoints` positive correlations, likely because VMs often rely on NSGs for traffic filtering.

Negative correlations (Lower misconfiguration risk):

`all_encryption_enabled` ($r = -0.33$) shows the strongest negative correlation. This is intuitive, indicating that templates with universal encryption are substantially less likely to contain misconfigurations, likely because they have already been improved or reviewed for security or operations settings. ($r = -0.15$) `all_managed_disks` suggests managed disk adoption is a protective factor, again likely reflecting more modern, well-reviewed templates. Template complexity variables (, ,) show near zero, `num_resources` `num_dependencies` `max_dependency_depth` suggesting infrastructure size alone does not strongly predict misconfiguration status.

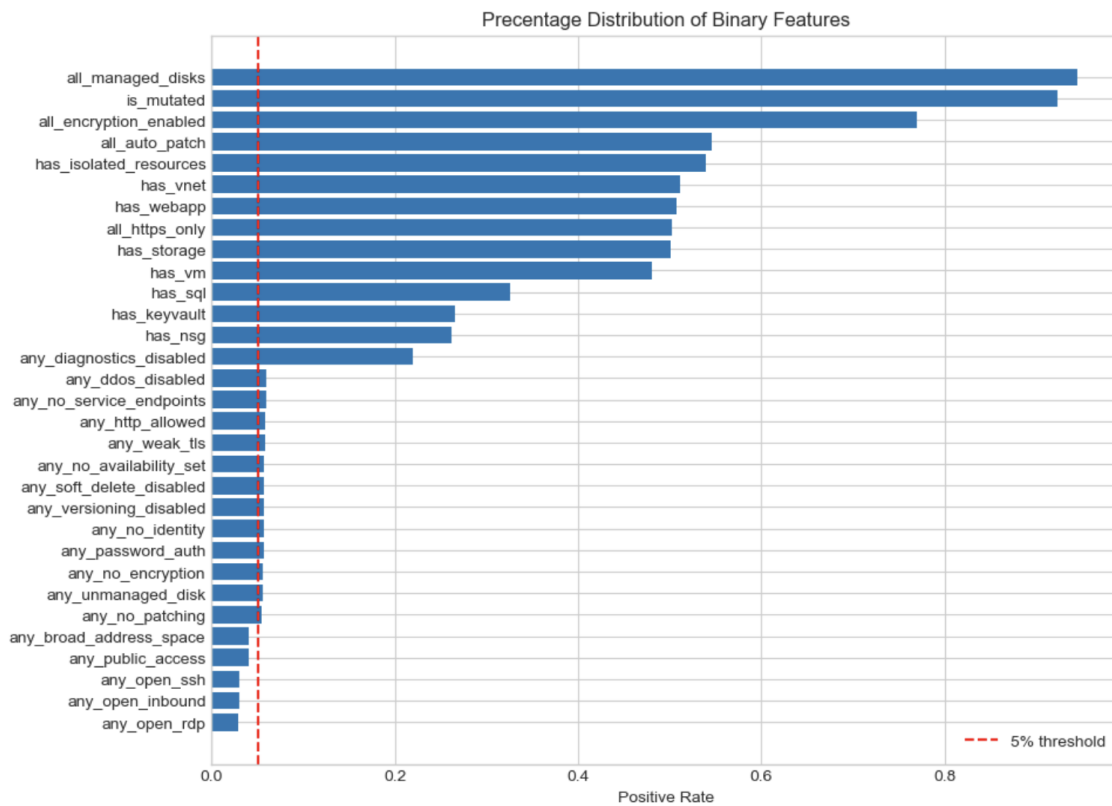


Figure 5: Figure 5

Figure 5: Percentage Distribution of Binary Features Figure

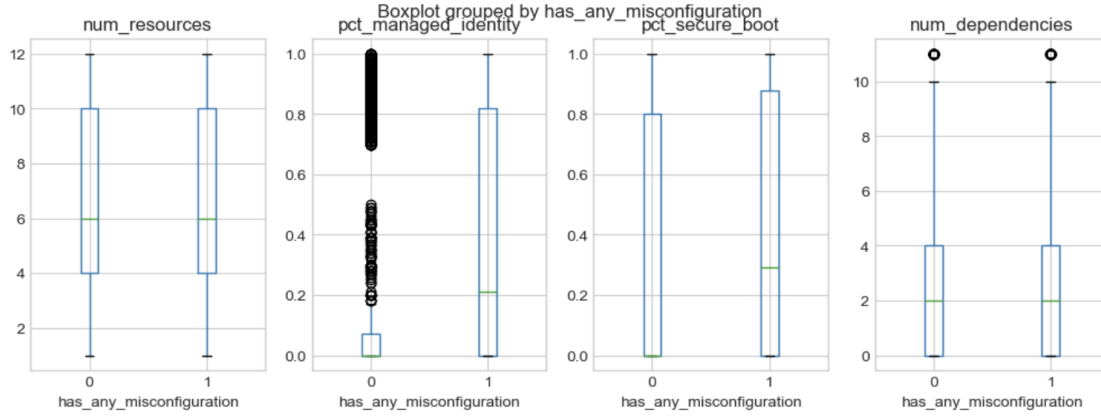


Figure 6: Figure 6

6: Key Independent Variable Relationships with Dependent Variable

3.7. Multicollinearity Assessment

This analysis used a standard correlation matrix for reviewing the multicollinearity of independent variables. In the case of any variable pairs exceeding a correlation threshold of 0.90, one variable from each pair would be removed to prevent instability in the logistic regression coefficients (Hosmer et al., 2013). Three pairs exceeded the threshold selected:

- `pct_secure_boot` and `pct_vtpm_enabled`: $r = 0.97$
- `all_managed_disks` and `any_unmanaged_disk`: $r = -1.00$
- `has_vm` and `pct_secure_boot`: $r = 0.92$

To address the findings, one variable from each pair was removed based on relevance to the research question.

`pct_vtpm_enabled` was removed in favor of `pct_secure_boot` since both measure VM security hardening but secure boot is more prevalent. Then `any_unmanaged_disk` was removed due to inverted collinearity with `all_managed_disks`, which are basically mirror images. Last, `pct_secure_boot` was retained over `has_vm` because the study is focused on security and management features where quantity in templating is likely a reproducible unit with similar characteristics, not a novel element contributing to security.

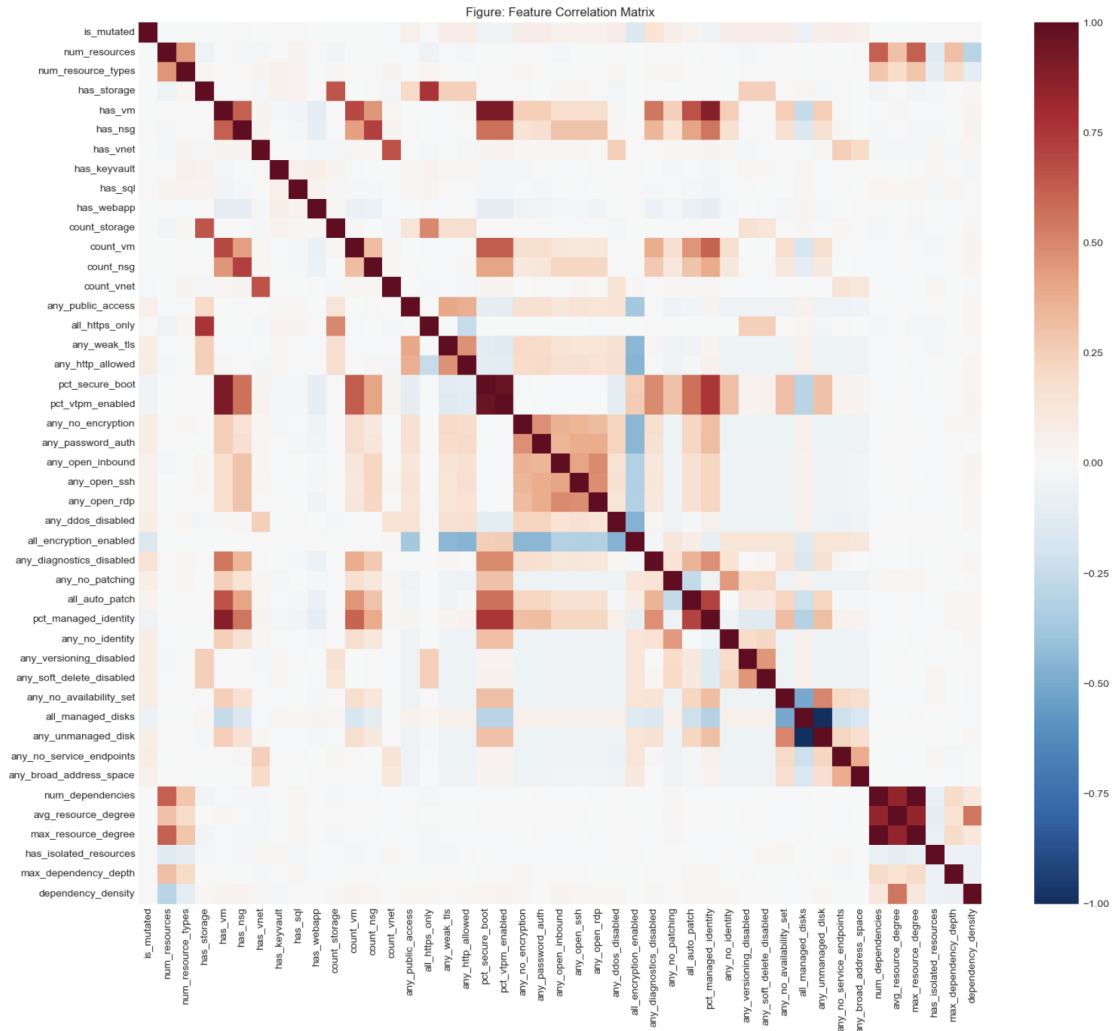


Figure 7: Figure 7

Figure 7: Feature Correlation Matrix

3.8. Train/Test Split and Feature Scaling

```
from sklearn.model_selection
import train_test_split
X = df_clean[final_iv_cols]
y = df_clean['has_any_misconfiguration']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42, s
print(f"Training set: {len(X_train):,} ({len(X_train)/len(X):.0%})")
print(f"Test set: {len(X_test):,} ({len(X_test)/len(X):.0%})")
```

result

Training set: 9,100 (70%) Test set: 3,900 (30%)

4. Analysis and Results

The analytical approach focused on for the study is logistic regression. Logistic regression was selected for its ability to produce interpretable odds ratios and quantify risk magnitude for these configurations. Logistic regression output transparency may allow security and operation teams to understand leading indicators causing a template to become flagged so they can prioritize remediation efforts. This section focuses on performing the logistic regression and then analyzing the results to validate the experiment design. Chi-square tests and one-way ANOVA are performed as tests to confirm mutation scenarios produced statistically different misconfiguration rates in compared to the baseline scenario and quantify the effect sizes.

4.1. Logistic Regression Implementation

Due to an issue found related to numeric instability while using statsmodels with the dataset (singular Hessian matrix according to research), the analysis uses scikit-learn LogisticRegression library with the L-BFGS solver as a remediation. The scikit-learn implementation applies L2 regularization by default ($C=1.0$), which mutes the effects of moderately correlated predictors identified in the VIF analysis, but retained due to the potential importance of the features.

```
from sklearn.linear_model import LogisticRegression
sklearn_model = LogisticRegression( max_iter=1000, solver='lbfgs', random_state=42, verbose=
sklearn_model.fit(X_train_scaled, y_train)

print("Logistic Regression Details: ")
print(f"Intercept: {sklearn_model.intercept_[0]:.6f}")
print(f"Coefficients shape: {sklearn_model.coef_[0].shape}")
print(f"Model accuracy (training): {sklearn_model.score(X_train_scaled, y_train):.4f}")
```

result

Logistic Regression Details:

Intercept: 4.501607 Coefficients shape: (40,) Model accuracy (training): 0.8943

4.2. Bootstrap Confidence Intervals

Since scikit-learn does not provide native p-values, an alternative approach called bootstrap sampling was used (Buisson, 2021). This approach aligns with the ASA recommendations that “methods emphasizing estimation over testing, such as confidence intervals provide robust inference” (Wasserstein & Lazar, 2016). A predictor is considered statistically significant when its interval exceeds 1.0. This method was used to compute 95% confidence intervals for each coefficient.

Of 40 features, 26 (65%) exceed 1.0 and are considered significant. The strongest protective factors were enabling encryption (OR = 0.07) and auto patching (OR = 0.08) and they reduce most misconfiguration odds by over 90%. The strongest risk factors were disabled versioning (OR = 10.89), and soft delete (OR = 10.47) which are 10x risks odds. The template composition variables relating to resource counts weren’t significant indicating that configuration choices matter more than a template complexity, at least in the study data that was used.

BOOTSTRAP CONFIDENCE INTERVALS (1,000 iterations) =====

* indicates 95% CI excludes zero (i.e. - significant)

=====			
Feature	Odds Ratio	95%	CI Significant
all_encryption_enabled	0.063710	[0.06, 0.07]	*
all_auto_patch	0.071483	[0.07, 0.08]	*
any_soft_delete_disabled	10.942725	[10.20, 11.95]	* any_versioning_disabled 10.804483
is_mutated	9.126037	[8.48, 9.66]	*
pct_secure_boot	6.272533	[4.35, 6.86]	*
any_no_identity	4.395892	[4.19, 5.03]	*
any_no_service_endpoints	3.498085	[3.44, 3.73]	*
all_managed_disks	0.302010	[0.28, 0.31]	*
any_no_availability_set	3.279055	[3.23, 3.58]	*
pct_managed_identity	3.124801	[2.54, 3.96]	*
any_broad_address_space	2.788658	[2.69, 2.90]	*
has_vnet	1.690006	[1.43, 1.80]	*
any_no_patching	1.561646	[1.47, 1.68]	*
any_diagnostics_disabled	1.524060	[1.27, 1.64]	*
any_http_allowed	1.439820	[1.37, 1.50]	*
any_no_encryption	1.435305	[1.33, 1.51]	*
any_password_auth	1.409277	[1.31, 1.46]	*
all_https_only	1.367216	[1.23, 1.37]	*
has_storage	0.742368	[0.70, 0.81]	*
any_ddos_disabled	1.264512	[1.21, 1.31]	*
any_open_ssh	1.264116	[1.17, 1.31]	*

```

any_open_inbound      1.249517      [1.16, 1.29]      *
any_weak_tls          1.230877      [1.22, 1.31]      *
any_open_rdp 1.223572 [1.19, 1.31] *
any_public_access 1.181415 [1.14, 1.21] *
count_storage 1.154087 [0.93, 1.15]
has_nsg 0.879819 [0.77, 1.05]
count_nsg 1.092324 [0.96, 1.28]
has_sql 1.073881 [0.92, 1.07]
num_dependencies 0.943207 [0.92, 1.14]
count_vnet 0.946789 [0.87, 1.11]
count_vm 0.951751 [0.88, 1.15] num_resources 1.039726 [0.86, 1.11] num_resource_types 1.0272
Significant predictors: 26 of 40

```

4.3. Model Performance

The logistic regression model was evaluated using scikit-learn’s metrics module on the test set (3,900 observations). The model achieved almost 90% accuracy (89.7%) on the test data, exceeding the baseline rate of 73.6% by 16.1% and the defined threshold of 78.6%. ROC AUC was 0.962, significantly above the 0.80 threshold specified in the hypothesis. Recall was 96.5%, indicating the model correctly identifies nearly all templates containing misconfigurations. Specificity was 70.5%, reflecting a trade-off that favors catching misconfigurations over minimizing false positives.

MODEL PERFORMANCE METRICS

```
=====
```

```

Accuracy: 0.899 (89.9%)
Precision: 0.916 (91.6%)
Sensitivity: 0.950 (95.0%)
Specificity: 0.757 (75.7%)
F1 Score: 0.932
ROC AUC: 0.963

```

4.4. Confusion Matrix Analysis

A confusion matrix is a table that visualizes model performance by comparing predicted classifications against actual outcomes. For binary classification, it produces four categories: true positives (correctly identified misconfigurations), true negatives (correctly identified clean templates), false positives (clean templates incorrectly flagged as misconfigured), and false negatives (actual misconfigurations that the model missed).

The confusion matrix shows 2,770 true positives and 727 true negatives, with 99 false negatives and 304 false positives. In the context of cloud security and operations screening:

- True Positives (2,770): Templates correctly identified to contain misconfigurations. These represent successful detections that would trigger remediation workflows.
- True Negatives (727): Clean templates correctly identified as proper configurations. These pass screening without unnecessary investigation.
- False Positives (304): Clean templates incorrectly flagged as misconfigured. While these create additional review workload, the cost is primarily time spent investigating templates that don't require remediation.
- False Negatives (99): Actual misconfigurations that the model failed to detect. These represent the highest-risk errors because misconfigured templates would proceed to production undetected.

The low false negative count (99 of 2,869 actual misconfigurations, or 3.5%) confirms the model rarely misses actual misconfigurations. This is a desirable property for security or operations screening where the cost of missing a vulnerability far exceeds the cost of investigating a false alarm.

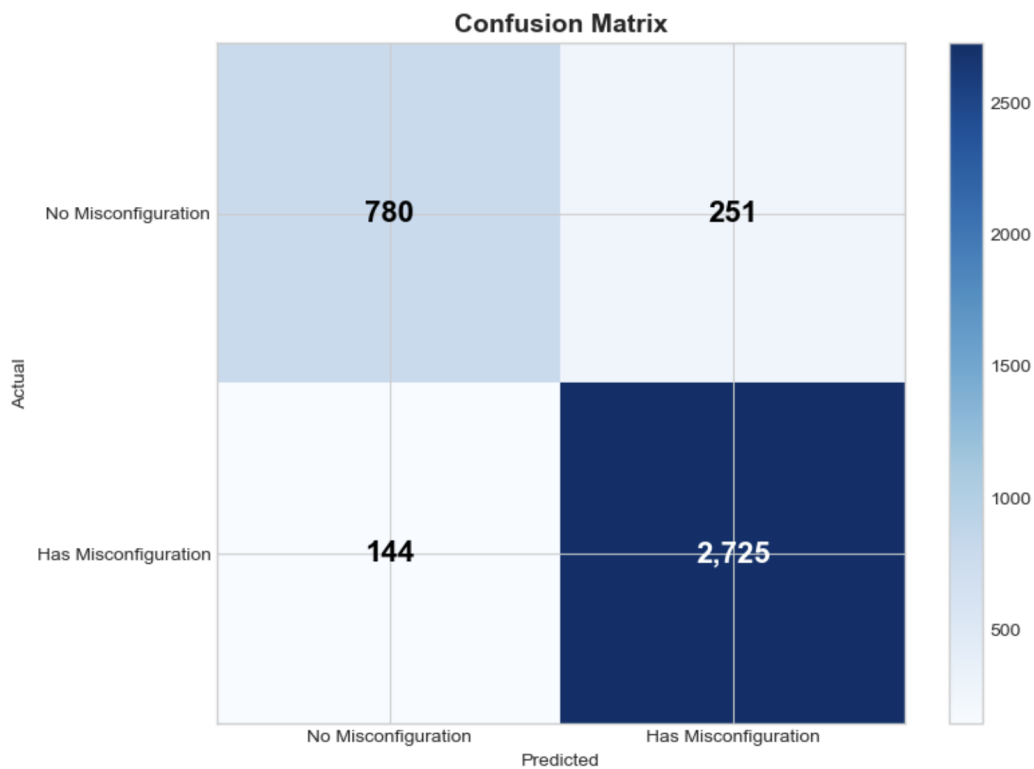


Figure 8: Figure 8

Figure 8: Confusion Matrix

4.5. ROC Curve Analysis

The ROC curve lets us visually inspect how well the model separates misconfigured from properly configured templates. A perfect classifier would hug the top-left corner, while a random guess follows the diagonal dashed line.

The curve hugs the top-left corner rather than following the diagonal (random chance), confirming the model's strong ability to discriminate between good and bad templates (AUC = 0.962).

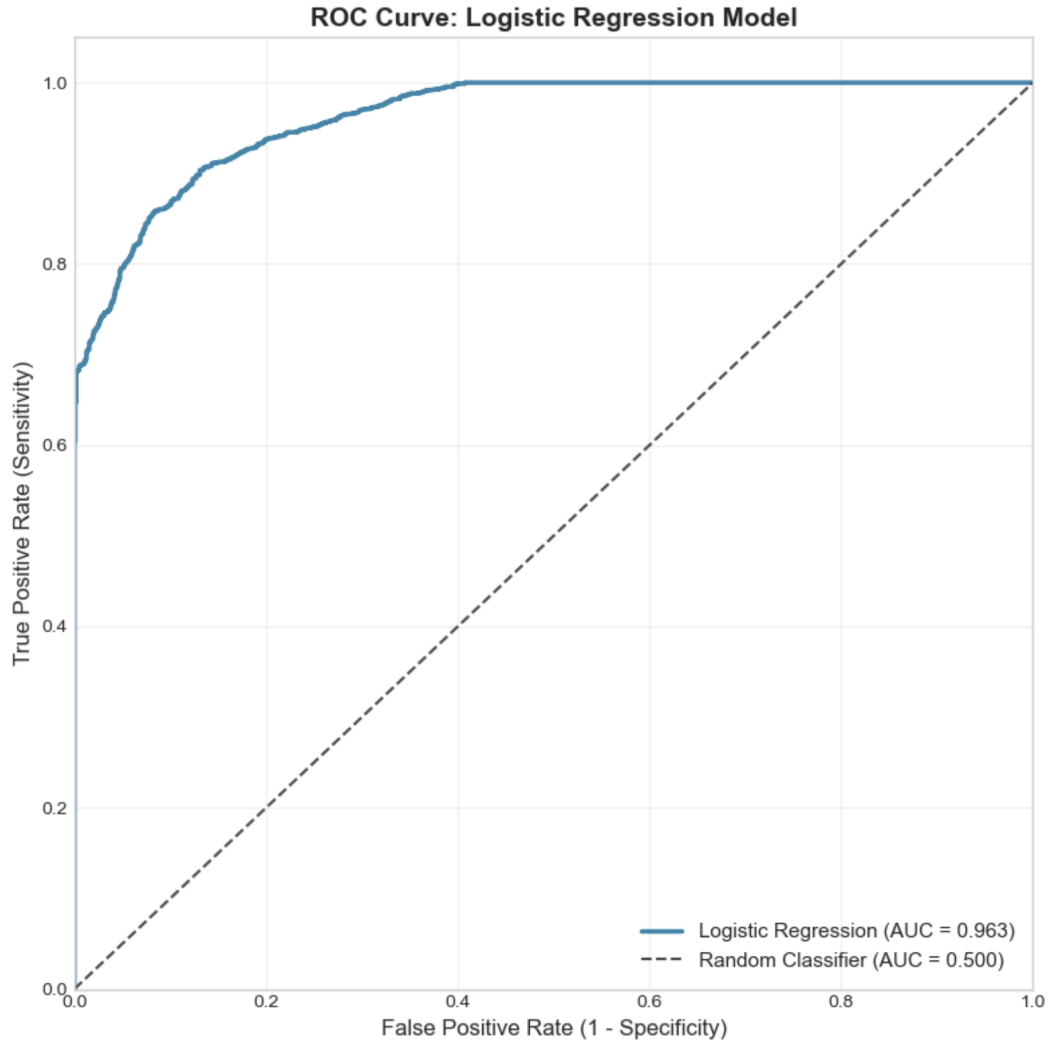


Figure 9: Figure 9

Figure 9: ROC Curve: Logistic Regression Model

4.6. Chi-square Test

Two methods were used to validate the experimental design: the chi-square test and contingency table. The chi-square test confirms that scenario category and misconfiguration status are dependent. The chi-square statistic ($\chi^2 = 5,265.80$) measures deviation from independence,

degrees of freedom ($df = 4$) reflect the number of category combinations, and the p-value (< 0.001) confirming this isn't due to chance. Cramér's V of 0.636 exceeds the 0.5 threshold for a large effect, meaning scenario categories strongly predict misconfiguration status.

Contingency Table:

has_any_misconfiguration	0	1
scenario_category combined	78	2922
control	1000	0
operational	1093	1907
reliability	1264	1736
security	2	2998
Chi-square statistic:	5265.80	
Degrees of freedom:	4	
p-value:	0.00e+00	
Cramer's V (effect size):	0.636	

4.7. Analysis of Variance (ANOVA)

One-way ANOVA was used to test whether mean misconfiguration counts differ across scenario categories. The F-statistic ($F = 3,330.97$) measures variance between groups relative to variance within groups, and the p-value (< 0.001) confirms the differences are statistically significant. Eta-squared ($\eta^2 = 0.506$) indicates that scenario category explains 50.6% of the variance in misconfiguration counts which is a large effect. The group means show a clear pattern: control scenarios average 0 misconfigurations, reliability and operational scenarios average around 1, while security and combined scenarios average over 3.5. This supports chi-square findings that the experimental design produces meaningful, measurable differences in misconfiguration severity across categories making the logistic regression results valid and trustworthy.

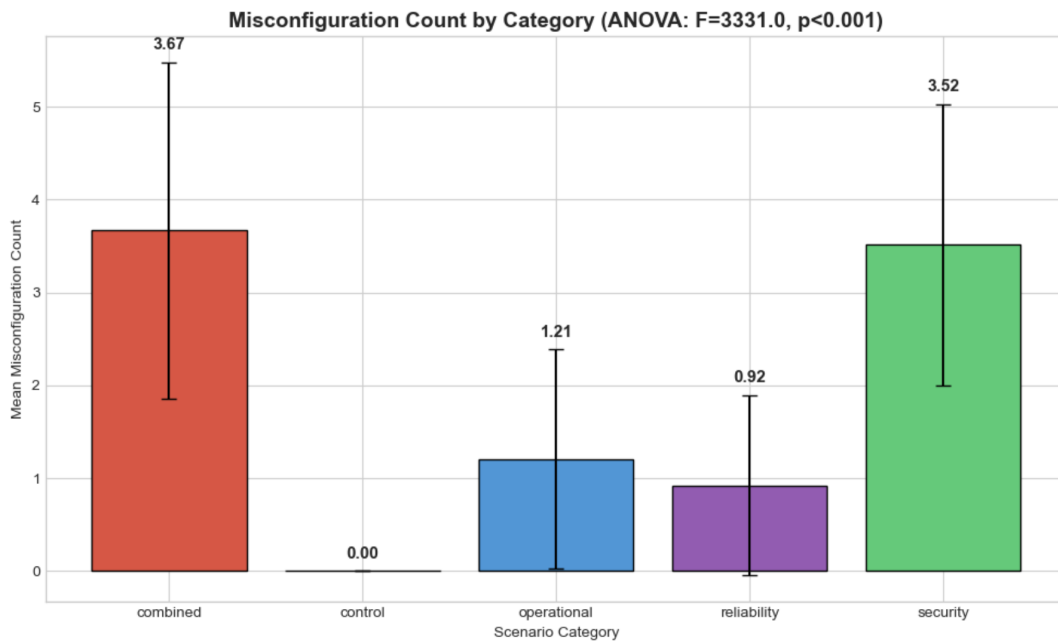


Figure 10: Figure 10

Figure 10: Misconfiguration Count by Category (ANOVA Results)

4.8. Residual Analysis

A Q-Q plot of the deviance residuals was produced to gauge normality. The Q-Q plot shows residuals deviating from normality, but this is not a concern—logistic regression does not assume normal residuals. The step-like pattern is typical for binary outcomes. Residuals are near zero (mean = 0.04) with no patterns indicating modeling issues.

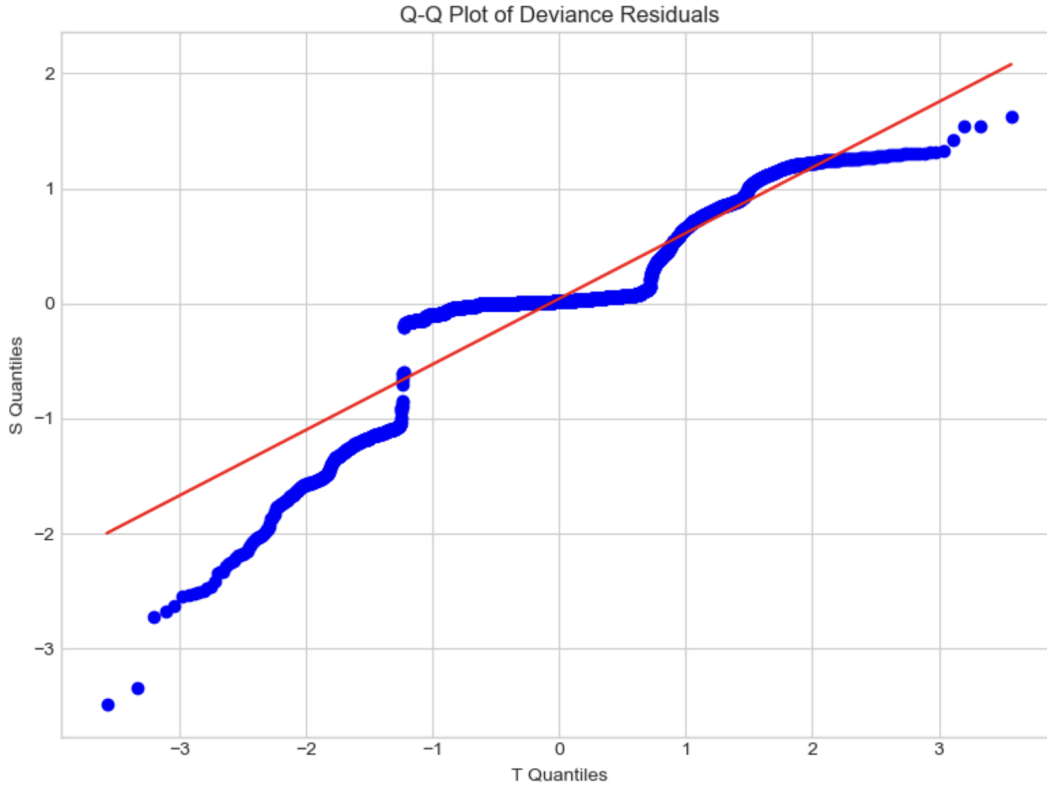


Figure 11: Figure 11

Figure 11: Q-Q Plot of Deviance Residuals

4.9. Hypothesis Test Result

The primary hypothesis stated that configuration properties would be significant predictors of misconfiguration status if the model achieved accuracy at least 5% above the baseline rate of 73.6% (threshold: >78.6%) with a ROC AUC greater than 0.80. The logistic regression model achieved 89.43% accuracy (a 15.8 percentage point improvement over baseline) and 26 of 40 features (65% of total features) had bootstrap confidence intervals that excluded 1.0. These all indicate the model has statistically significant predictive relationships. Based on these results, the null hypothesis is rejected and we assert that template-level configuration properties are statistically significant predictors of infrastructure misconfiguration status.

5. Discussion and Conclusions

This study demonstrates that Azure resource configuration properties are statistically significant predictors of infrastructure misconfigurations. The logistic regression model achieved almost 90% accuracy (89.7), substantially exceeding the baseline rate of 73.6% and the pre-specified hypothesis threshold of 78.6%.

The analysis identified 26 statistically significant predictors out of 40 features examined (65%). Key findings include:

1. Encryption and patching are the strongest protective factors – Enabling encryption across all resources (OR = 0.07) and automatic patching (OR = 0.08) reduce misconfiguration odds by over 90%.
2. Data protection settings are critical – Disabling versioning (OR = 10.89) or soft delete (OR = 10.47) increases risk approximately 10x.
3. Configuration choices matter more than complexity – Template composition variables (resource counts, dependency metrics) weren't significant predictors, indicating that how resources are configured matters more than how many a template contains.
4. VM security settings show strong effects – Secure boot percentage (OR = 5.20), managed identity absence (OR = 4.81), and managed disk usage (OR = 0.30) all significantly predict misconfigurations.
5. Synthetic dataset methodology is effective – The controlled mutation scenarios produced meaningful differences in misconfiguration rates, validated by chi-square and ANOVA tests, supporting the experimental design.

5.1. Limitations

The limitation identified during this study is the analysis only focuses on Microsoft Azure configurations as represented by their configuration language “ARM” (Azure Resource Manager). The odds ratios and predictors may or may not generalize well on Amazon (AWS), Google (GCP), or other managed provider environments due to unique configuration schemas, operational and security controls. The modeling approach, tools and technologies may require additional abstractions or separate risk models per platform.

5.2. Recommendations

After completing the analysis of this model, there are four classes of recommendations that could further enhance this body of work by improving the model and benefit organizations implementing a similar strategy:

Risk-Tiered Remediations: Organizations using this approach should implement a risk-tiered remediation strategy based on the quantified odds ratios. Configurations with the highest impact should be addressed first: enabling encryption and automatic patching (Severity 1 – Critical) reduces misconfiguration odds by over 90%, while enabling soft delete and versioning (Severity 2 – High) prevents a 10x risk increase. This evidence-based prioritization allows security teams to allocate limited resources where they’ll have the greatest effect.

Address Class Imbalance: Future iterations of this model should address the class imbalance (73.6% positive cases) that biases predictions toward misconfigurations, resulting in lower recall for clean templates (71%). Techniques such as SMOTE oversampling, class weighting, or adjusted decision thresholds could improve specificity without sacrificing sensitivity.

Expand Dataset Diversity: To enhance model generalizability, future work should expand the dataset to include more diverse templates and real-world samples. The current synthetic dataset may not capture all configuration patterns seen in practice. Collecting anonymized templates from various organizations, cloud providers, and industries would provide a richer training set.

Building Customer Tools: Any organization implementing a system like this not only needs the machine learning model, but also a customer-facing tool to operationalize the insights. A web dashboard or CLI tool that scans templates, applies the model, and generates prioritized remediation reports would make the findings actionable. Integrating with CI/CD pipelines to automatically screen templates before deployment could prevent misconfigurations from reaching production.

5.3. Future Research Directions

1. Multi-Cloud Extension: Apply the CRUX methodology to AWS CloudFormation templates and Terraform

configurations to develop cross-platform risk models. This would enable organizations with multi-cloud deployments to apply consistent risk quantification across their entire infrastructure.

2. Interaction Effect Analysis: Extend the logistic regression model to include interaction terms between

high-risk predictors and identify configuration combinations that present risk beyond their individual effects.

3. Alternative Modeling Techniques: Explore ensemble methods (Random Forest, XGBoost) or neural

networks to capture potential non-linear relationships that logistic regression cannot model.

4. Open-Source Project Expansion: The CRUX project is open source and available on GitHub. Future

research could focus on expanding the project to allow community contributions of real-world templates, additional mutation scenarios, and integration with CI/CD pipelines for continuous risk assessment.

References

Buisson, F. (2021, November 4). Ditch p-values. Use Bootstrap confidence intervals instead. Towards Data Science. <https://towardsdatascience.com/ditch-p-values-use-bootstrap-confidence-intervals-instead-bba56322b522>

Center for Internet Security. (2023). CIS Microsoft Azure Foundations Benchmark v2.0.0. <https://www.cisecurity.org/benchmark/azure>

Gartner. (2019, October 10). Is the cloud secure? <https://www.gartner.com/smarterwithgartner/is-the-cloud-secure>

Hair, J. F., Anderson, R. E., Tatham, R. L., & Black, W. C. (1995). Multivariate data analysis (3rd ed.). Macmillan.

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression (3rd ed.). Wiley.

IBM. (2023). Cost of a Data Breach Report 2023.

McKinney, W. (2017). Python for Data Analysis (2nd ed.). O'Reilly Media.

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python.

Wasserstein, R. L., & Lazar, N. A. (2016). The ASA's statement on p-values: Context, process, and purpose. *The American Statistician*, 70(2), 129-133. <https://doi.org/10.1080/00031305.2016.1154108>

Wheeler, B. (2025). CRUX: Cloud Resource mUtation eXaminer (Version 1.0) [Computer software]. GitHub. <https://github.com/floatingsidewall/CRUX>